Modern Cloud Computing

Lecture 2: Concurrency and Parallelism

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- Concurrency and Parallelism Basics
- Distributed Computing Basics
- Parallel Architectures
- Distributed Systems and Scalability

Concurrency and Parallelism Basics

Concurrency

Cloud computing is fundamentally concurrent.

- Many users run workloads at the same time.
- One application may use many processes, threads, VMs, containers, or functions.

Concurrency vs. Parallelism

Concurrency describes multiple activities that are active during the same time period.

Parallelism describes multiple activities literally executing at the same time on multiple processing units.

Concept	Main emphasis
Concurrency	Coordination, interaction, interference
Parallelism	Reducing completion time using multiple processors

Communication Is the Price of Concurrency

Concurrent entities often need to communicate to complete a shared task.

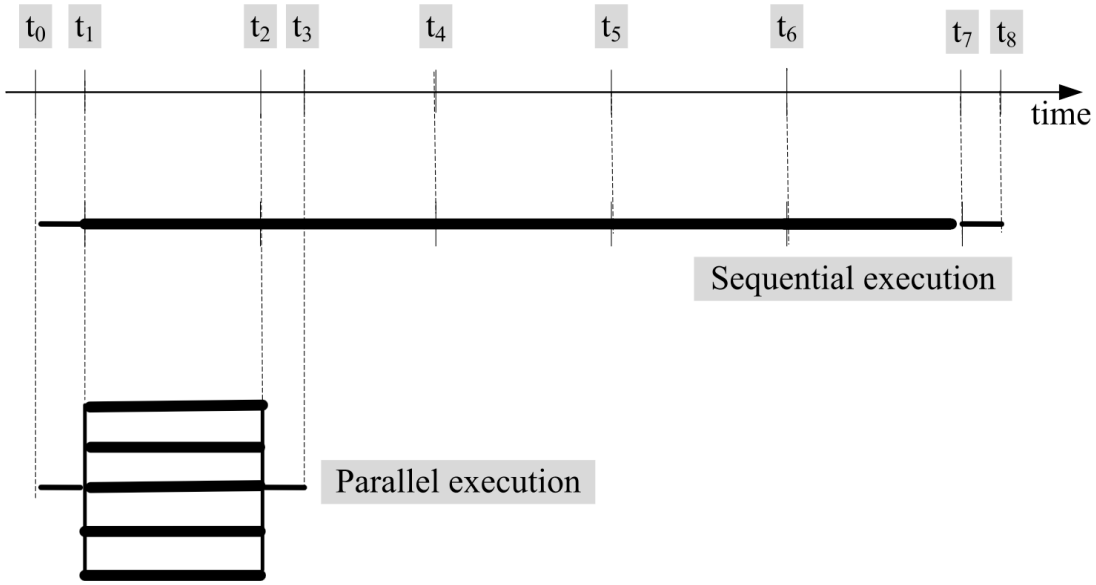
Communication introduces several costs:

- Synchronization
- Waiting time
- Message delay
- Lost or delayed messages
- Coordination overhead

Motivating Example: Parallel Image Processing

A simple data-parallel workload:

- Convert **5 million images** from one format to another.
- Split the input into 5 groups of 1 million images.
- Run 5 workers concurrently.
- If workers do not communicate, speedup can approach 5×.



(Source: Cloud Computing: Theory and Practice, 2nd Edition, Fig. 3.3)

Speedup

Speedup measures how much faster a parallel execution is compared with a sequential execution.

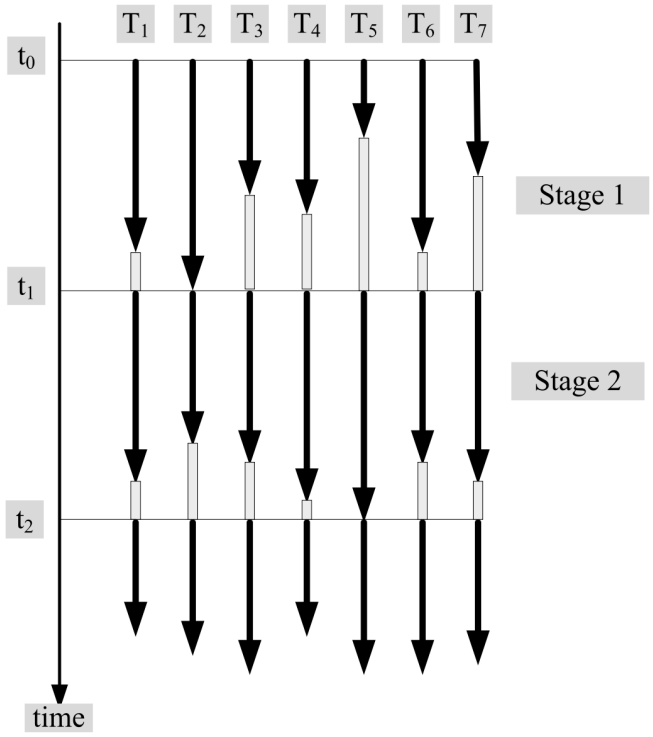
$$S = \frac{T_{sequential}}{T_{parallel}}$$

For the image-processing example:

$$S \approx 5$$

Why not always perfect?

- Initialization and termination phases
- Communication overhead
- Scheduling overhead
- Synchronization waits



Barrier Synchronization

Sometimes a computation has multiple stages.

A thread or process cannot move to the next stage until all parallel activities in the current stage finish.

- Fast workers finish early.
- Slow workers become stragglers.
- Everyone waits at the barrier.

(Source: Cloud Computing: Theory and Practice, 2nd Edition, Fig. 3.4)

Coarse-Grained vs. Fine-Grained Parallelism

Type	Description	Cloud relevance
Coarse-grained	Large independent blocks of work	Batch jobs, MapReduce, parameter sweeps
Fine-grained	Small compute bursts with frequent communication	Tightly coupled simulations, distributed ML training

Coarse-grained workloads usually scale better in cloud environments. (Example: the parallel image processing task we showed earlier)

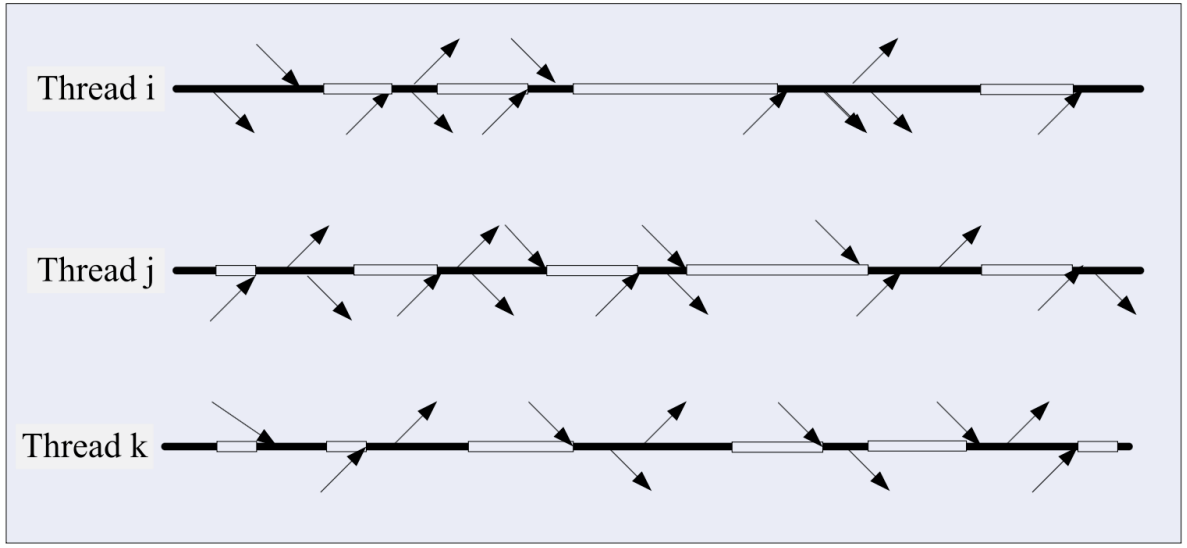
Fine-grained workloads are more sensitive to network latency and synchronization overhead.

Fine-Grained Parallelism

Fine-grained parallelism alternates between:

- Short bursts of computation
- Waiting for messages or synchronization

This can lead to poor speedup, because processors spend significant time blocked.



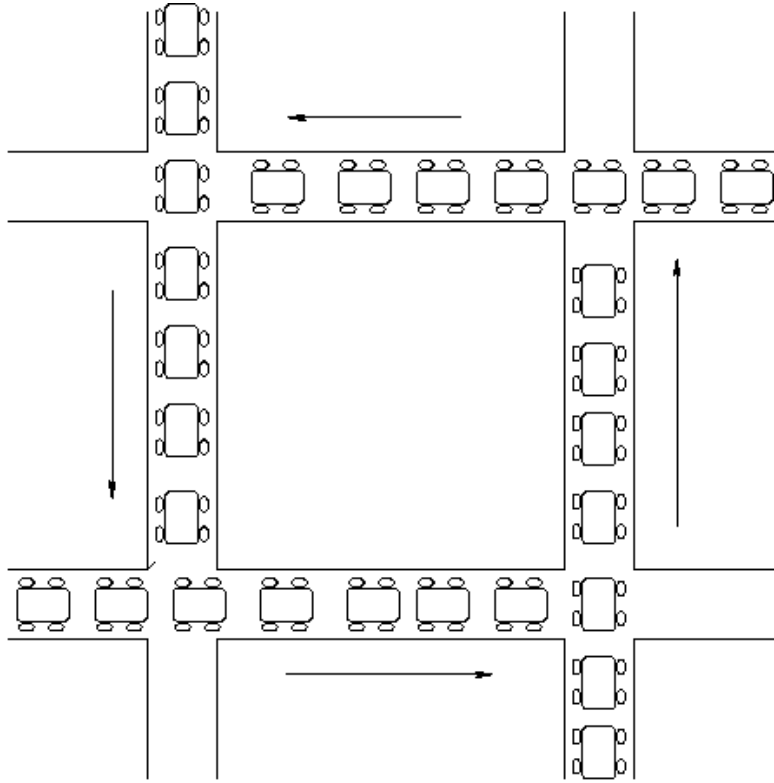
(Source: Cloud Computing: Theory and Practice, 2nd Edition, Fig. 3.5)

Deadlock: A Coordination Failure

A deadlock occurs when a set of activities wait forever for resources held by each other.

Traffic analogy:

- Intersections are shared resources.
- Cars enter without coordination.
- Each direction blocks another direction.
- No one can move.



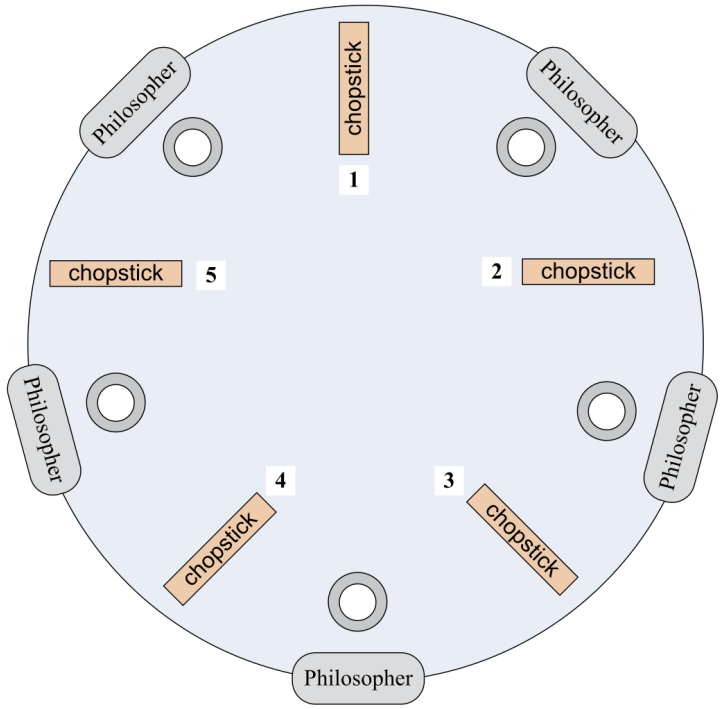
(Source: CSCI.4210 Operating System, RPI)

Dining Philosophers

The dining philosophers problem

Naive strategy:

1. Each philosopher picks up one chopstick.
2. Each waits for the second chopstick.
3. Everyone waits forever.



(Source: Cloud Computing: Theory and Practice, 2nd Edition, Fig. 3.2)

Avoiding Deadlock with Resource Ordering

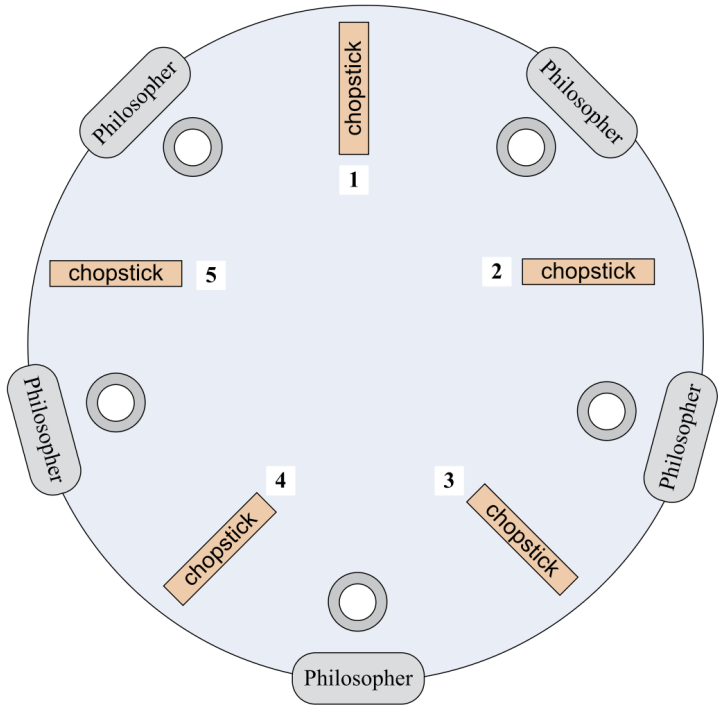
Dijkstra's solution uses an ordering discipline:

1. Assign a partial order to resources.
2. Require resources to be requested in order.
3. Do not allow a unit of work to hold unrelated resources simultaneously.

For dining philosophers:

- Number the chopsticks.
- Each philosopher picks the lower-numbered chopstick first.
- Then picks the higher-numbered chopstick.

Cloud analogy: enforce consistent ordering when acquiring distributed locks or resources.



Computational Models

Abstract model used in the analysis of algorithms

- Turing Machine
- Finite State Machine
- Random Access Machine

...

Bulk-Synchronous Parallel Model (BSP)

Goal: Free programmer from managing memory, communication and low-level synchronization

BSP: v virtual parallel processors running on $p \leq v$ physical processors. It includes:

- Processing elements and memory components
- A router involved in message exchanges
- Synchronization acting every L units of time

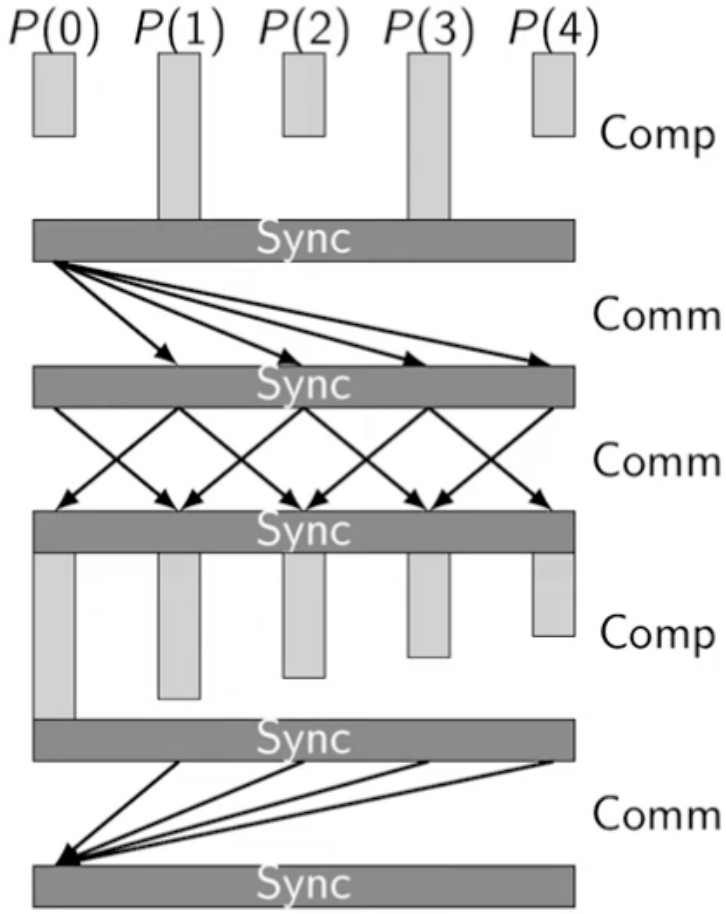
BSP (Continued)

BSP is a model for parallel computation organized into **supersteps**.

A **task**: combination of local computations and message exchanges.

A **superstep**: execution unit allocated L time units and consisting of multiple tasks.

1. Each component assigned a task
2. After L units of time, global check performed (barrier sync)
    - If *Current superstep has been completed by all components*: initiate next superstep
    - Else: Allocate another L units of time



BSP Superstep Intuition

A BSP program alternates between local work and global coordination.

```
Superstep 1: compute locally + send messages + synchronize
Superstep 2: compute locally + send messages + synchronize
Superstep 3: compute locally + send messages + synchronize
...
```

Cost is given by:

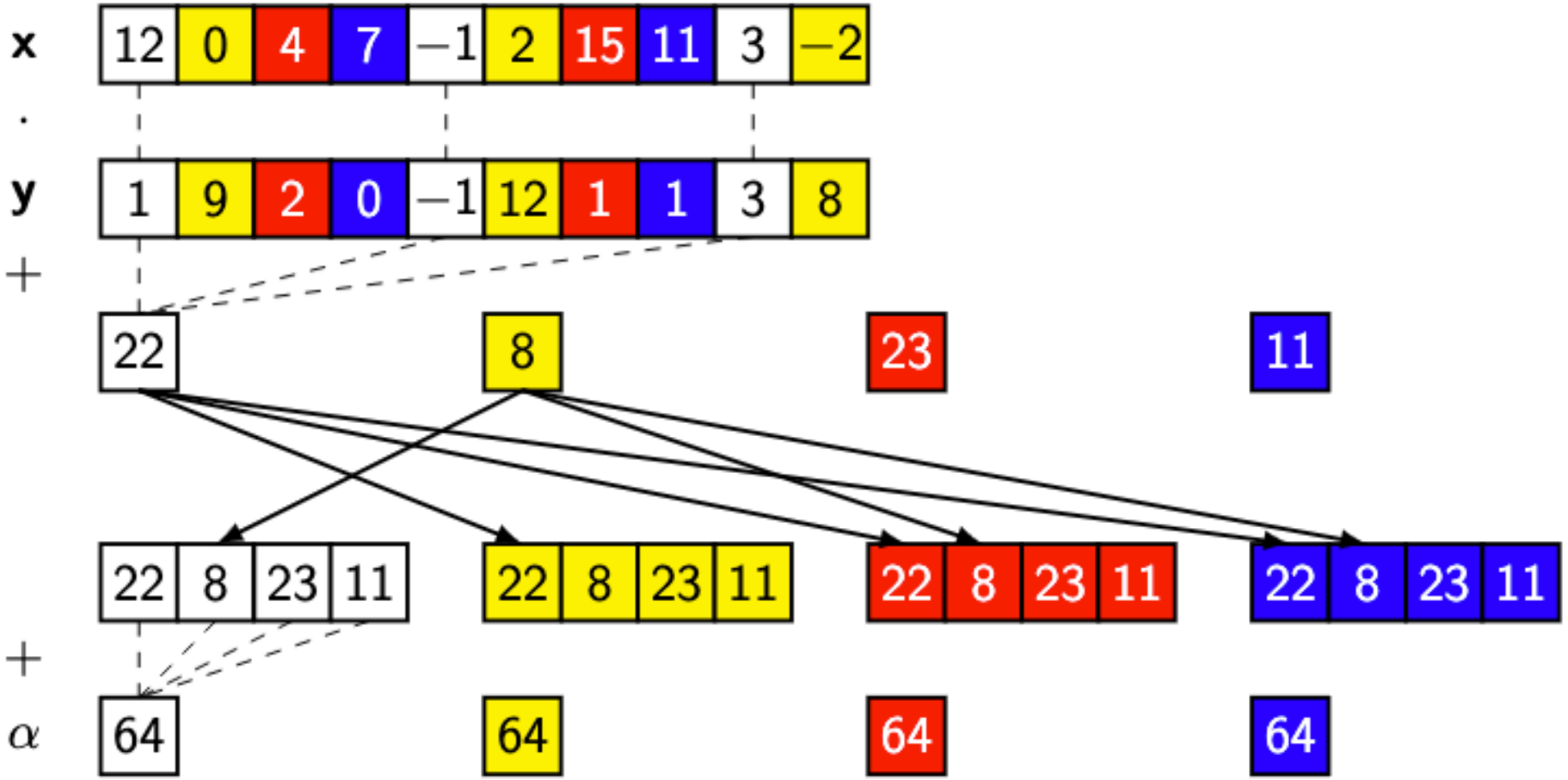
Communication: $hg + l$, where h: maximum number of data words sent/received by a processor; g: time cost per data word; l: sync cost

Computation: $w + l$, where w: maximum flop computation time

Cost of mixed superstep: $w + gh + l$

An example: Inner Product Calculation

BSP Example: Inner Product Calculation



BSP Example: Inner Product Calculation (Cont'd)

```
# Superstep (0)
αs := 0
for i := s to n – 1 step p do
    αs := αs + x_i y_i

# Superstep (1)
for t := 0 to p – 1 do
    put αs in P(t)

# Superstep (2)
α := 0
for t := 0 to p – 1 do
    α := α + α_t
```

BSP Example: Inner Product Calculation (Cont'd)

Superstep 0 (Computation): $2\left\lceil \frac{n}{p} \right\rceil + l$

Superstep 1 (Communication): $p - 1$ relation, $(p-1)g + l$

Superstep 2 (Computation): $p + l$

Total: $2\left\lceil \frac{n}{p} \right\rceil + (p-1)g + p + 3l$

Distributed Computing Basics

Process and Thread State

A process is a program in execution.

A thread is a lightweight execution unit scheduled by the operating system.

The **state** of a process/thread is the information needed to restart it after suspension.

A state may include:

- Program counter
- Registers
- Stack
- Memory mappings
- Open resources
- Communication state

Local State and Global State

In a distributed system, each process has its own local state.

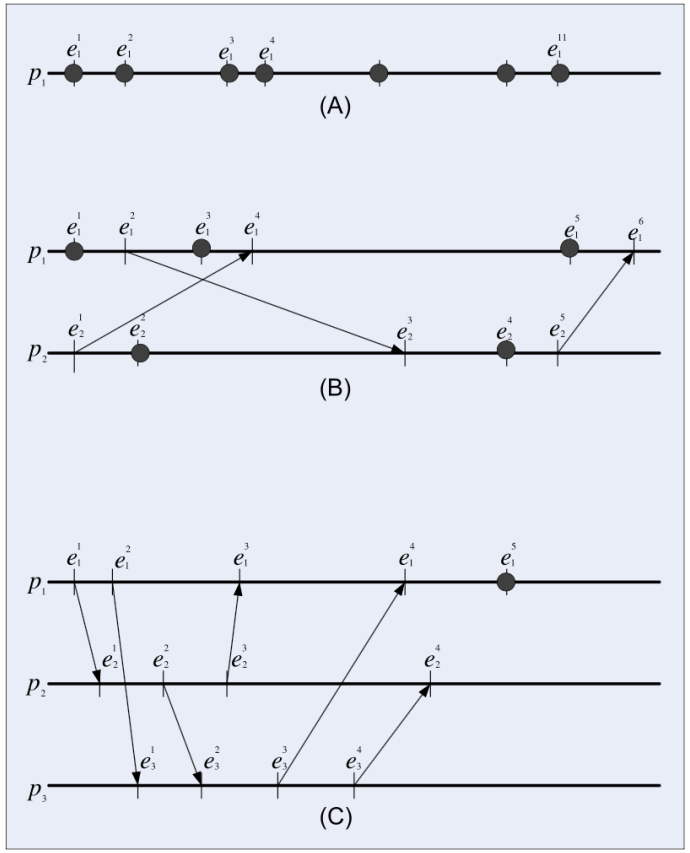
The **global state** includes:

- The local state of every process $\sigma_i^{j_i}$
- The state of communication channels

For processes $p_1, p_2, \ldots, p_n$, global state is:

$$\sum^{(j_1, j_2, \ldots, j_n)} = (\sigma_1^{j_1}, \sigma_2^{j_2}, \ldots, \sigma_n^{j_n})$$

- Solid points: local events
- Arrows: communication events



(Source: Cloud Computing: Theory and Practice, 2nd Edition, Fig. 3.12)

Why Global State Is Hard

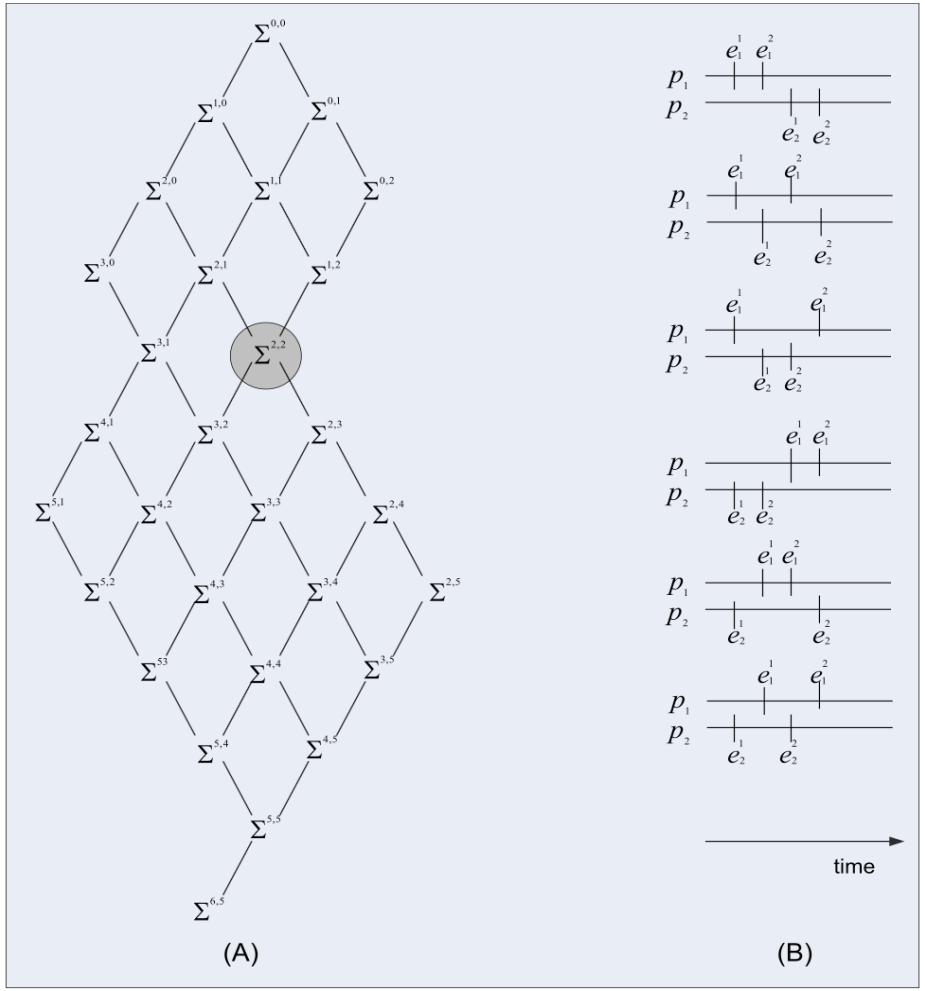
Different processes observe events in different orders.

Problems:

- Message delays
- No shared physical clock
- Partial failures
- Many possible event interleavings
- Difficult debugging

In a system with q threads, the number of path to reach $\sum^{(n_1,n_2,\ldots,n_q)}$ from $\sum^{(0,0,\ldots,0)}$ is a huge number:

$$N_p^{(n_1,n_2,\ldots,n_q)} = \frac{(n_1 + n_2 + \ldots + n_q)!}{n_1! n_2! \ldots n_q!}$$



(Source: Cloud Computing: Theory and Practice, 2nd Edition, Fig. 3.13)

Communication Protocols and Coordination

A protocol is a finite set of messages exchanged to coordinate processes.

Protocols are needed for:

- Mutual exclusion
- Leader election
- Checkpointing
- Replication
- Consensus
- Distributed transactions
- Load balancing

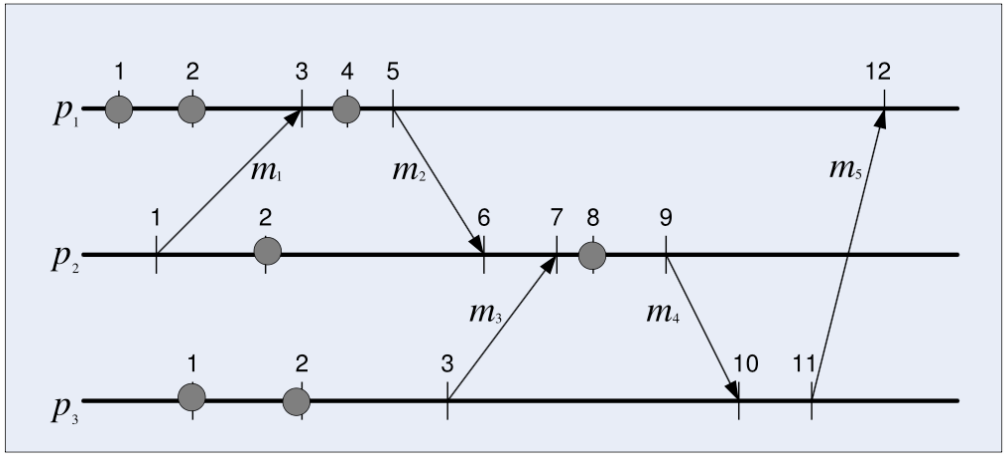
Cloud implication: many cloud services are protocol machines hidden behind APIs.

Logical Clocks

Physical clocks are hard to rely on in distributed systems.
Logical clocks assign event timestamps based on causal order.

Lamport's "happened-before" relation:

- Events in the same process are ordered by local sequence.
- Sending a message happens before receiving it.
- The relation is transitive.



Logical clocks help reason about causality without relying on perfectly synchronized physical clocks.

$$LC(e) = \begin{cases} LC + 1, & \text{if } e \text{ is a local event or a } \text{send}(m) \text{ event}, \\ \max(LC,\ TS(m)) + 1, & \text{if } e = \text{receive}(m). \end{cases}$$

Stronger guarantees simplify programming but increase coordination cost. -->

Runs, Cuts, and Consistent Cuts

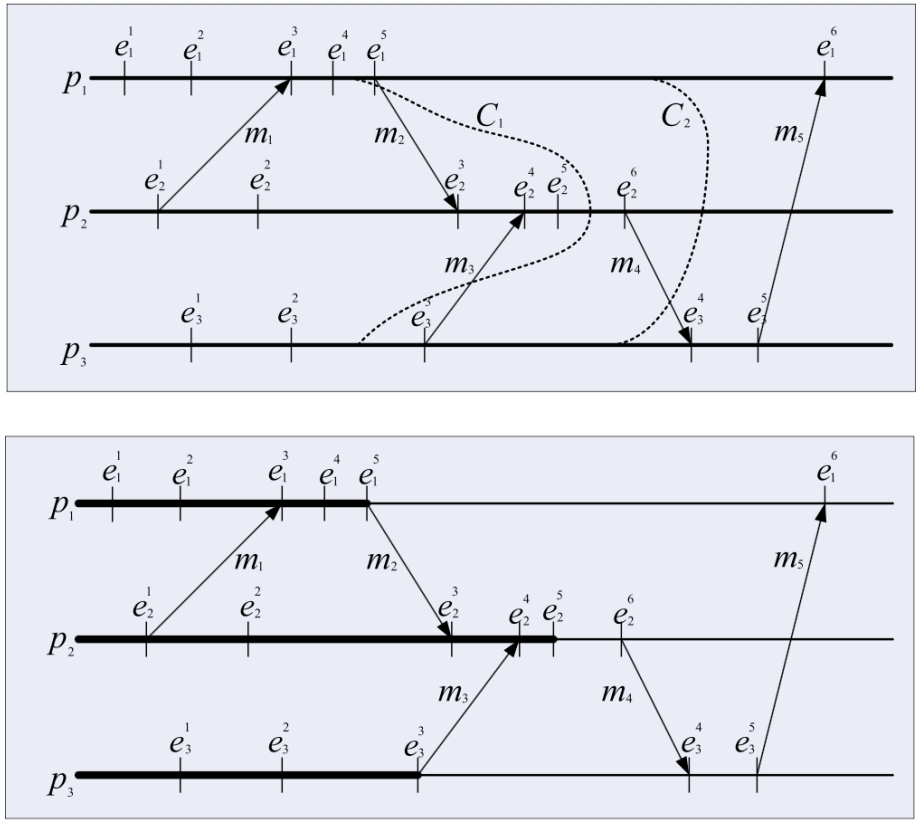
A **run** is one possible sequence of events in a distributed computation.

- A run is valid if it is consistent with global history.

A **cut** selects one local state from each process.

- A cut is **consistent** if it does not violate causality:
- If a receive event is included, the corresponding send event must also be included.
- The cut should represent a possible global state.

Top fig: C_1 is inconsistent as it contains e_2^3 and e_2^5 but not e_1^5 and e_3^3, violating causality. C_2 is consistent.
Bottom fig: Smallest consistent cut containing e_2^5 is $\{e_1^1, e_1^2, e_1^3, e_1^4, e_1^5, e_2^1, e_2^2, e_2^3, e_2^4, e_2^5, e_3^1, e_3^2, e_3^3\}$



(Source: Cloud Computing: Theory and Practice, 2nd Edition, Fig. 3.19 & 3.20)

Distributed Snapshots and Checkpointing

Long-running cloud computations need fault tolerance.

Checkpointing saves state so work can restart after failure.

A distributed snapshot must capture a consistent global state:

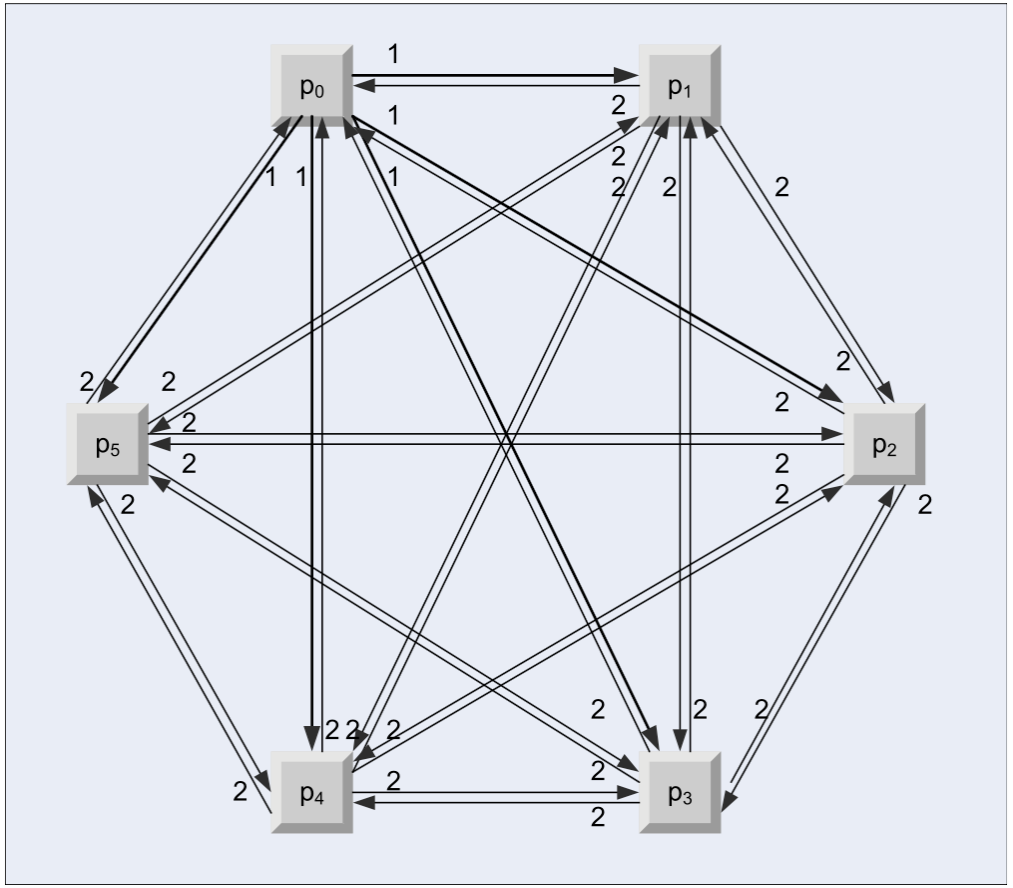
- Process states
- Messages in communication channels
- Dependencies between events

Why it matters: without consistent checkpoints, recovery may produce impossible states.

Snapshot protocol

Each "take snapshot" message crosses each channel exactly once, every process contributes to the snapshot

- In step 0, p_0 sends to itself the "take snapshot" message.
- In step 1, process p_0 sends five "take snapshot" messages labeled (1).
- In step 2, each of the five processes, p_1, p_2, p_3, p_4, and p_5, sends a "take snapshot" message labeled (2) to every other process.



Consensus

Consensus means a group of processes agree on one value, even when failures occur.
Used for:

- Leader election
- Replication
- Metadata management
- Distributed locks
- Configuration management

Consensus protocl example: Paxos

Consensus is difficult because systems may have:

- Message delays
- Crashes
- Network partitions
- Incomplete information

Load Balancing

Load balancing distributes work across resources.

Goals:

- Improve utilization
- Reduce response time
- Avoid hotspots
- Handle dynamic workloads
- Improve availability

Cloud load balancing can occur at many levels:

- DNS / global routing
- Front-end request routing
- VM/container placement
- Task scheduling
- Storage request distribution

Parallel Architectures

Parallel Processing

Parallel processing solves large problems by splitting them into smaller pieces and solving them concurrently.

It requires advances in:

- Algorithms
- Programming models
- Compilers and runtimes
- Interconnection networks
- Computer architecture
- Performance monitoring

Cloud computing makes parallel resources broadly accessible.

Data-Level Parallelism

Data-level parallelism partitions data into chunks and processes chunks concurrently.

Examples:

- Search an object in many images
- Scan many records for a string
- Process large logs
- Transform many files
- Train on data partitions

This is common in cloud computing because many enterprise workloads are data parallel.

Same Program, Multiple Data (SPMD)

In SPMD:

- Multiple workers run the same program.
- Each worker processes a different data partition.
- Workers may synchronize or exchange messages.

SPMD fits many cloud workloads:

- Map tasks
- Batch analytics
- Simulations over parameter sets
- Distributed ML data-parallel training

Task-Level Parallelism

Task-level parallelism decomposes a job into tasks that may differ from one another.

Examples:

- Workflow pipelines
- Independent optimization trials
- FPGA design exploration
- Microservices handling different parts of a request
- Cloud jobs with many scheduled tasks

Task-level parallelism is often coordinated by a scheduler.

Communication: Shared Memory vs. Message Passing

Model	How processes communicate	Strength	Limitation
Shared memory	Read/write common memory	Fast on one machine	Hard to scale; difficult debugging
Message passing	Send/receive messages	Scales across machines	Higher latency; protocol complexity

Cloud-scale systems mostly rely on message passing.

Parallel Architectures

Parallelism appears at many levels:

- Bit-level parallelism
- Instruction-level parallelism
- SIMD/vector parallelism
- Thread-level parallelism
- Task-level parallelism
- Distributed parallelism across machines

Cloud platforms combine many of these layers.

SIMD and Vector Processing

SIMD means **Single Instruction, Multiple Data**.

The same operation is applied to many data elements at once.

Good for:

- Multimedia processing
- Image processing
- Linear algebra
- Scientific computing
- Machine learning kernels

SIMD exposes data parallelism directly in hardware.

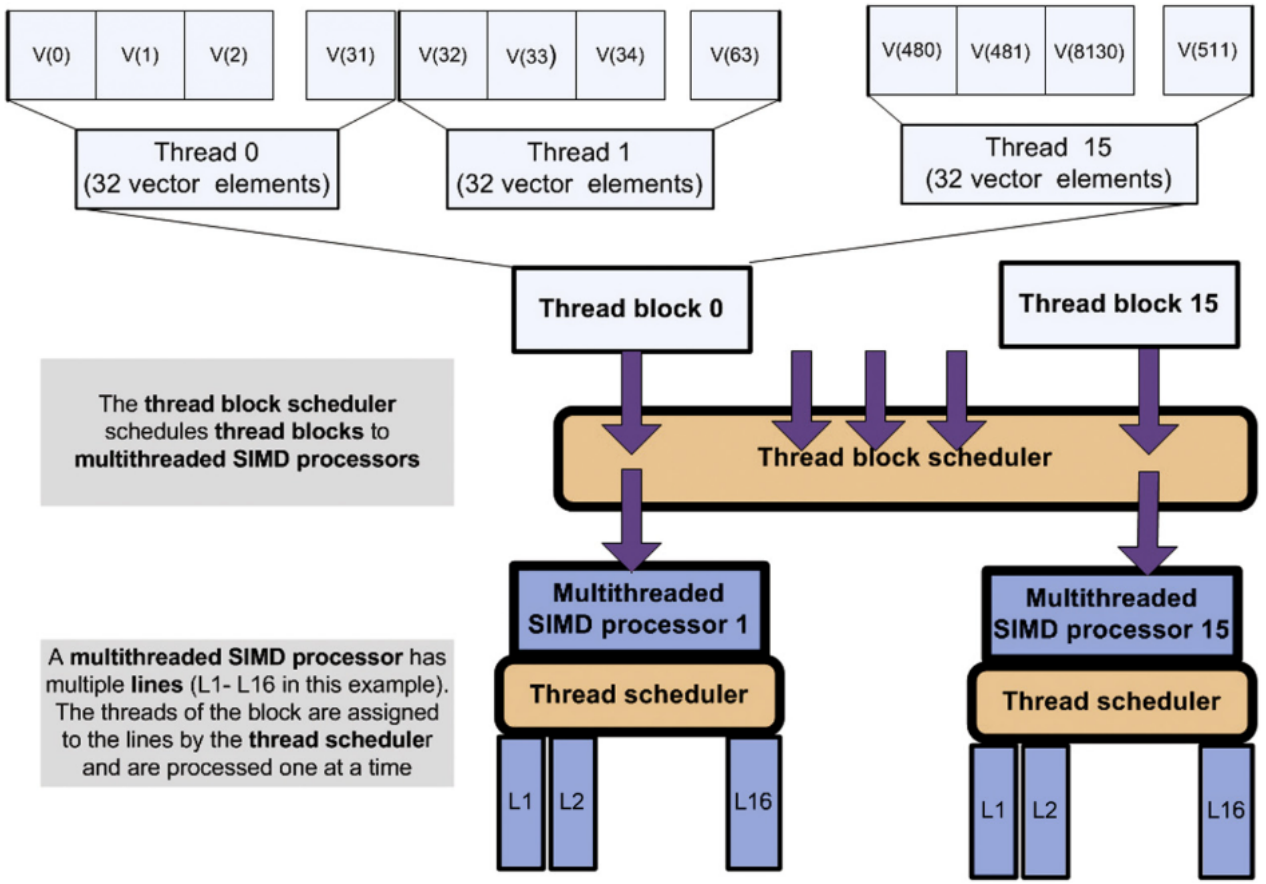
GPUs

Graphics Processing Units were originally designed for graphics workloads.
They are now widely used for general-purpose data-parallel computation.

GPU strengths:

- Many lightweight threads
- High memory bandwidth
- SIMD-style execution
- Strong performance for matrix/vector operations

Cloud providers offer GPU instances because many workloads need accelerator-based parallelism.



(Source: Cloud Computing: Theory and Practice, 2nd Edition, Fig. 4.3)

Amdahl's Law

Amdahl's Law limits speedup when part of a computation is sequential.

If α is the non-parallelizable fraction:

$$S \leq \frac{1}{\alpha}$$

Examples:

Sequential fraction	Maximum speedup
10%	10×
5%	20×
1%	100×

Key idea: the sequential part eventually dominates.

Gustafson's Law and Scaled Speedup

Amdahl's Law assumes a fixed problem size.

Gustafson's Law: What if problem size allowed to change?

$$S(N) = N - \alpha(N - 1)$$

N: number of parallel processes

This is important for cloud computing because users often scale out to process larger datasets, not only to solve the same dataset faster.

Distributed Systems and Scalability

Distributed Systems

A distributed system consists of multiple autonomous components that cooperate through communication.

Cloud systems are distributed by design:

- Data centers contain many machines.
- Regions and availability zones distribute infrastructure.
- Services are decomposed into many components.
- Failures are expected and isolated.

Key challenge: make many unreliable parts look like one reliable service.

System Modularity

Modularity decomposes a system into components.

Benefits:

- Easier development
- Easier testing
- Fault isolation
- Independent evolution
- Reuse of components

Costs:

- Interfaces must be carefully designed.
- Cross-component communication creates overhead.
- Hidden dependencies can hurt reliability.

Soft vs. Enforced Modularity

Soft modularity: modules are separated by software conventions.

Enforced modularity: modules are forced to interact only by sending and receiving messages.

Layering and Hierarchy

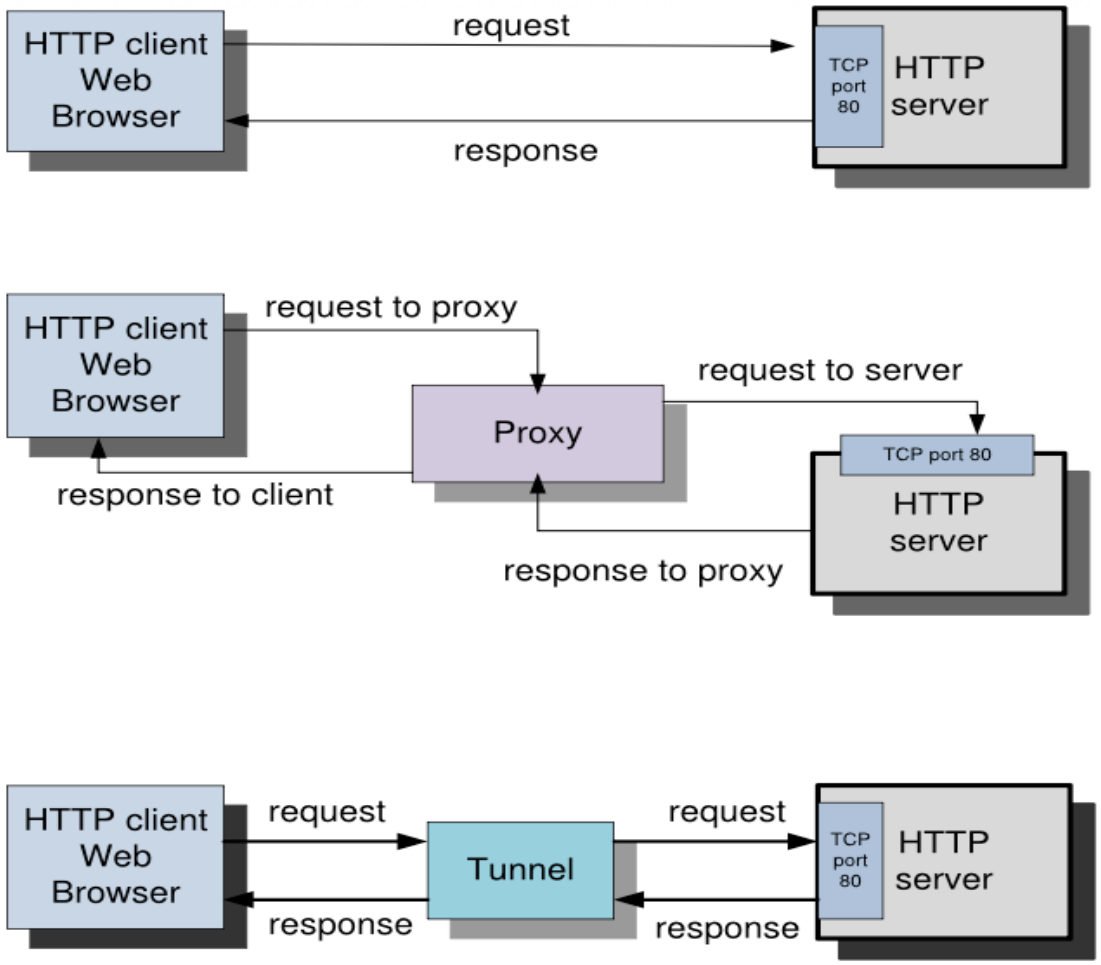
Layering organizes systems into levels of abstraction.

Benefits:

- Hides complexity
- Enables portability
- Supports independent evolution

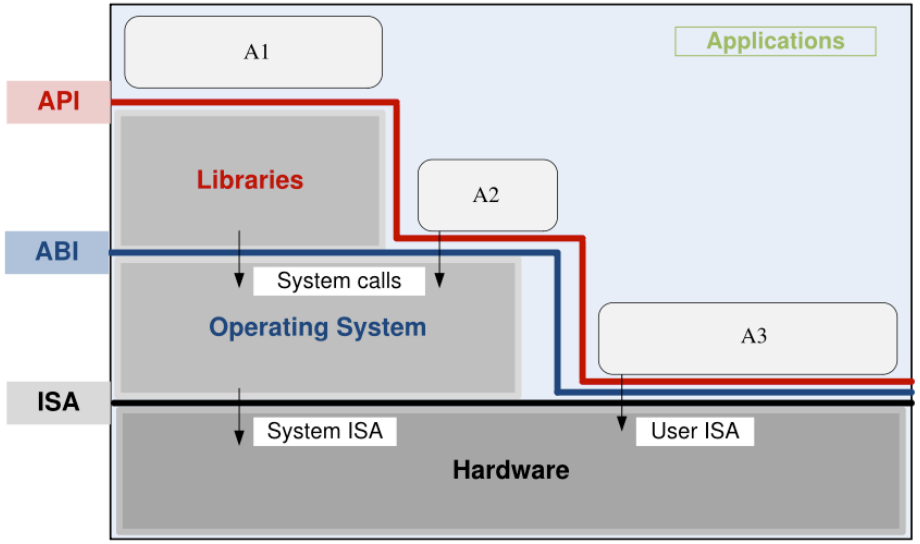
Risk:

- Too many layers can reduce performance and obscure failure causes.



(Source: Cloud Computing: Theory and Practice, 2nd Edition, Fig. 4.8)

Virtualization



(Source: Cloud Computing: Theory and Practice, 2nd Edition, Fig. 4.9)

Virtualization creates a logical view of resources that differs from the physical view.
Cloud virtualization can abstract:

- CPU
- Memory
- Storage
- Network
- Operating systems
- Application environments

Virtualization enables multi-tenancy, isolation, migration, elasticity, and resource pooling.

Peer-to-Peer (P2P) Systems

Peer-to-peer systems distribute responsibility across peers instead of relying on centralized servers.

Characteristics:

- Peers can act as both clients and servers.
- Resources are contributed by many nodes.
- Membership may be dynamic.
- Decentralized lookup and routing may be needed.

Cloud systems are usually provider-managed, but P2P ideas influence distributed storage and decentralized coordination.

Large-Scale Systems

Large-scale cloud systems face problems that small systems can ignore:

- Partial failures are common.
- Monitoring data is incomplete or delayed.
- Local decisions may create global instability.
- Debugging is difficult.
- Rare corner cases happen regularly at scale.

Scalability

A system is scalable if it can handle increasing load by adding resources.

Scalability depends on:

- Parallelizable workload fraction
- Communication overhead
- Synchronization overhead
- State management
- Scheduling policy
- Bottleneck resources
- Fault tolerance design

Summary

- Concurrency is about coordination; parallelism is about speed.
- Communication, synchronization, and shared resources create the hardest problems.
- Data-parallel and coarse-grained workloads usually fit the cloud best.
- Amdahl's Law explains why speedup is limited; Gustafson's Law explains why cloud scale is still valuable.
- Virtualization and modularity make cloud infrastructure manageable, but add complexity.

Questions?